

High-Performance Architectural Patterns for WebAssembly-Driven Grand Strategy Game User Interfaces

The Zero-Copy Memory Bridge: WebAssembly, SharedArrayBuffer, and the Main Thread

Grand strategy games represent a unique and punishing computational challenge within the browser environment. These simulations require tracking tens of thousands of individual entities—representing economic units, military divisions, diplomatic relations, and dynamic demographic populations—each mutating multiple times per second within a strict 16.6-millisecond frame budget for a 60 FPS target. Traditionally, bridging a high-frequency, deterministic state machine written in C++ or Rust to a web-based user interface required serializing engine data into JSON or passing heavily copied objects through the browser's `postMessage` API. This serialization incurs massive overhead, frequently triggering the V8 JavaScript engine's mark-and-sweep garbage collection (GC) cycles. When the GC runs to clear thousands of intermediate objects, it induces fatal frame drops, resulting in user interface stuttering that destroys the real-time simulation experience. The modern architectural solution relies on the `SharedArrayBuffer` (SAB) primitive, which establishes a lockless, zero-copy memory bridge between WebAssembly (WASM) linear memory and the JavaScript execution context.

When a WebAssembly module is compiled and instantiated, its underlying memory model is initialized as a `WebAssembly.Memory` object. By explicitly backing this memory object with a `SharedArrayBuffer`, the linear memory space becomes concurrently accessible to both the WASM simulation environment (typically running within an isolated `Web Worker`) and the main browser thread.¹ This paradigm shift means that game state data never undergoes a copy operation across the thread boundary. Instead, JavaScript `TypedArray` views—such as `Int32Array`, `Float32Array`, or `Float64Array`—are draped directly over the shared memory buffer, allowing the UI presentation layer to read the deterministic game state directly from system RAM.³

However, concurrent memory access necessitates robust synchronization paradigms to prevent race conditions, dirty reads, and read-tearing, wherein the UI might read a 64-bit float while the WASM engine is halfway through updating its bytes. JavaScript's `Atomics` API provides the necessary primitives for this synchronization without requiring the expensive context switches associated with traditional operating system mutexes.⁵ Historically, developers utilized `Atomics.wait()` to halt a thread until a specific memory index changed. However, this method is strictly prohibited on the main browser thread to prevent malicious or poorly optimized code from permanently freezing the user interface.⁶ The modern, production-grade solution, introduced in

recent ECMAScript specifications (such as ES2024), relies on `Atoms.waitAsync()`.⁸ This non-blocking API allows the main thread to listen for a state tick from the WASM engine, yielding a Promise that resolves the precise moment `Atoms.notify()` is invoked by the simulation thread.⁸ This architecture ensures the UI only reads from the `SharedArrayBuffer` when the simulation has definitively completed its deterministic tick, establishing a safe, lockless, read-only state representation.

One historical bottleneck in this zero-copy architecture has been the handling of dynamic text. Even when passing raw numerical data efficiently via a `SharedArrayBuffer`, rendering text dynamically (such as character names, dynamic province titles, or localized strings) required decoding UTF-8 byte sequences into JavaScript's immutable UTF-16 string representations via the `TextDecoder` API.¹¹ This decoding process inherently allocates fresh memory on the JavaScript heap, inevitably triggering the V8 garbage collector and undermining the benefits of a pure zero-copy pipeline. To resolve this, the 2025 and 2026 WebAssembly standards have heavily pushed the `stringref` and JavaScript String Built-ins proposals.¹¹ These proposals allow WASM to natively handle and pass opaque references to JavaScript strings without the overhead of copying or decoding bytes back and forth.¹³ By enabling WebAssembly implementations to internally represent strings in encoding-agnostic formats and pass them directly as DOM-ready references, GC pauses related to high-frequency string rendering (such as updating a thousand-row diplomatic ledger) are effectively eliminated, allowing for zero-copy string transfer.¹⁴

Synchronizatio n Mechanism	Thread Blocking	Main Thread Support	GC Allocation Profile	Primary Use Case in Engine Architecture
<code>postMessage (JSON Clone)</code>	Non-blocking	Yes	High (Deep Copy)	Low-frequency configuration changes; Initialization.
<code>SharedArrayBu ffer</code>	N/A (Direct Access)	Yes	Zero Allocation	High-frequency continuous state reading.
<code>Atoms.wait()</code>	Blocking	No	Zero Allocation	Synchronizing background Web

				Workers.
Atomics.waitAsync()	Non-blocking (Promise)	Yes	Minimal (Promise object)	Alerting main thread to a completed WASM tick without freezing UI.
JS String Built-ins (stringref)	N/A	Yes	Zero Allocation	Rendering dynamic localization and procedural names.

Off-Main-Thread UI Calculation and Primitive Mutation Pipelines

Even with a zero-copy memory architecture providing instantaneous access to the game state, the sheer computational effort of calculating the visual differences (diffing) required to update a complex DOM tree can rapidly overwhelm the main thread. Browsers are inherently single-threaded for rendering, layout calculation, and JavaScript execution. If JavaScript blocks the main thread to calculate how a complex economic outlier should change based on new state data, the browser cannot physically paint the frame, leading to severe degradation of the Interaction to Next Paint (INP) metric and catastrophic UI jank.¹⁸ To maintain the illusion of a seamless, native-feeling application, grand strategy engines employ an Off-Main-Thread (OMT) architecture, which completely decouples UI business logic from the physical UI rendering pipeline.

In a production-grade OMT architecture tailored for strategy games, the Web Worker does not solely run the WASM simulation; it also hosts the reactive state tree or the Virtual DOM (VDOM) reconciliation algorithms.¹⁸ When the WASM engine completes a simulation tick, the UI logic residing within the worker queries the SharedArrayBuffer. It executes the necessary algorithms to determine the exact visual changes required—for instance, calculating that a single text node must change from "1450 Ducats" to "1452 Ducats", or that a specific CSS class must be toggled on a military unit icon.²² Rather than generating complex JavaScript objects to represent these changes, the worker serializes these exact visual deltas into a highly optimized instruction set.

Because Web Workers operate in an isolated global scope and lack direct access to the DOM

API (window or document), these calculated "diffs" must be communicated back to the main thread.²⁰ The most performant pattern is to pass highly minimal, primitive mutation instructions. Rather than passing an entire state object tree, the worker transmits a flattened, optimized array of numeric opcodes and primitive values (e.g., ``). This message is transmitted via the `postMessage` API. While `postMessage` inherently incurs a copying cost via the structured clone algorithm, passing primitive arrays of integers and short strings generates extremely low serialization overhead.¹⁸ As long as the serialized payload remains under 10KB, the communication latency is negligible, and crucially, the main thread is freed entirely from performing business logic, diffing, or state reconciliation.¹⁸

The main thread is thereby reduced to a "dumb" execution environment. It maintains an internal dictionary mapping numeric node IDs to live, cached DOM elements. Upon receiving the mutation instruction array from the worker, a tight loop simply decodes the opcodes and executes the exact DOM API calls requested (`node.textContent = ...` or `node.classList.add(...)`). This instruction pipeline ensures that the main thread spends less than 1 to 2 milliseconds per frame actively manipulating the DOM, leaving the remainder of the 16.6ms budget exclusively for the browser's graphics pipeline to recalculate styles, perform repaints, and composite the final image onto the screen.²⁵

This architectural separation profoundly improves the engine's resilience against hardware variance. On lower-end mobile devices or ultra-thin laptop CPUs, calculating a 10,000-element UI diff might take 30 milliseconds. If executed on the main thread, the frame rate would immediately plummet to 30 FPS, and user interactions like scrolling or hovering would freeze. By executing this diff in a background worker, the game simulation continues uninterrupted, and the main thread remains free to process user scrolling, CSS hover states, and input event collection at a flawless 60 FPS.¹⁸ The UI merely delays its data update by one or two frames, seamlessly masking the computational latency behind fluid interactivity.¹⁹

Mitigating Layout Thrashing and Leveraging CSS Hardware Acceleration

When a game engine passes massive arrays of mutation instructions to the main thread, the specific sequence in which the DOM is manipulated becomes the ultimate determining factor of rendering performance. A grand strategy UI frequently involves deep, highly nested DOM trees containing thousands of nodes. When JavaScript alters a DOM element, the browser invalidates the current layout tree and schedules a recalculation. If JavaScript attempts to read a geometric property (such as `offsetHeight`, `clientHeight`, `getBoundingClientRect`, or `getComputedStyle`) after a mutation has been made but before the frame has actually painted, the browser is forced to halt script execution and synchronously calculate the entire page layout to return the correct value. This phenomenon, known as Layout Thrashing or Forced Synchronous Layout, is fatal to high-frequency rendering and will instantly drop engine performance well below acceptable thresholds.²⁷

To bypass layout thrashing when thousands of UI data points mutate simultaneously, game engines must employ strict read/write batching frameworks, frequently modeled after patterns like FastDOM. The architectural rule is absolute: all DOM reads (measurements) must occur at the very beginning of the requestAnimationFrame (rAF) cycle, utilizing the cached, valid layout from the previous frame. All DOM writes (mutations) are then deferred, grouped, and executed sequentially at the end of the rAF cycle.²⁷ By mathematically guaranteeing that no read operation ever follows a write operation within the same frame execution block, synchronous reflows are entirely eliminated.²⁷

Furthermore, engine architects must understand the performance ceilings of standard HTML rendering engines—such as Blink (Chrome/Edge) and Gecko (Firefox)—which are heavily bound by which specific CSS properties are mutated.³² The browser's critical rendering path follows a strict pipeline: DOM/CSSOM creation → Render Tree → Layout → Paint → Composite.³⁴ Modifying geometric dimensions (width, height, padding, margin) or absolute positioning (top, left) triggers the Layout pipeline, forcing the CPU to recalculate the geometry of the target element, its children, and potentially its siblings. Modifying aesthetics (color, background, box-shadow) triggers the Paint pipeline, bypassing geometry calculations but still consuming heavy CPU cycles to rasterize pixels.

To achieve a stable 60 FPS while animating heavy, dynamic elements—such as floating tooltip windows containing nested graphs, massive draggable world maps, or panning technology trees—DOM manipulation must bypass both the Layout and Paint phases entirely. This is achieved via CSS Hardware Acceleration. By utilizing specific CSS properties like transform: translate3d(x, y, z) and opacity, combined with the CSS optimization hint will-change: transform, the browser is forced to elevate the targeted DOM element to its own dedicated GPU layer.³¹ When the WebWorker sends new coordinates to pan a map or move a unit token, mutating the translate3d property does not trigger layout or paint on the CPU. Instead, it triggers the Composite step. The GPU simply shifts the pre-rasterized texture within Video RAM (VRAM). This critical distinction allows modern browser engines to handle thousands of moving UI elements with near-zero CPU overhead, a strict requirement for rendering complex strategy interfaces overlaying WebGL canvases. Notably, while Chromium's Blink engine excels at raw V8 JavaScript execution, Mozilla's Gecko engine, utilizing the Stylo/Servo CSS parsing engine, frequently demonstrates superior performance in handling deeply nested CSS recalculations during complex DOM mutations.³³

Browser Pipeline Phase	Triggered By	Performance Impact	Executing Hardware	Mitigation Strategy in Engine

Layout (Reflow)	width, height, top, offsetHeight reads	Severe (CPU Blocking)	CPU (Main Thread)	Batch all writes via FastDOM. Strictly avoid dynamic geometric changes during animations.
Paint	color, background, box-shadow	Moderate	CPU (Rasterizer)	Minimize DOM size. Disable drop-shadows on moving elements.
Composite	transform: translate3d, opacity	Minimal (Optimal)	GPU	Elevate interactive and animating elements to dedicated layers via will-change: transform.

High-Frequency Reactivity, UI Virtualization, and Render Throttling

While vanilla JavaScript, paired with strict DOM batching, provides maximum theoretical performance, maintaining a sprawling grand strategy game UI containing hundreds of views, panels, and interaction states in vanilla code results in unsustainable architectural complexity and technical debt. Thus, the adoption of reactive UI frameworks is an industry standard. However, traditional Virtual DOM (VDOM) frameworks face severe, fundamental limitations under extreme frame-budget pressure.³⁶

React's runtime architecture is fundamentally misaligned with high-frequency, deterministic tick engines. When the WASM state updates 60 times a second, React's Virtual DOM must initiate a reconciliation phase: it constructs a new JavaScript object tree representing the intended UI state, diffs it against the previous tree, and calculates the minimum necessary patch.³⁶ For dense grand strategy UIs containing thousands of elements (such as an inventory matrix, an economic ledger, or an empire outliner), this process demands continuous, massive memory allocation and object traversal. This memory churn invariably triggers the V8 engine's GC cycles.³¹ When the GC pauses execution to sweep discarded VDOM nodes, the main thread halts entirely, leading to temporal desynchronization where the UI stutters visibly while the

WASM simulation continues unimpeded in the background.³⁷ While techniques like aggressive memoization, `useMemo`, and immutable data structures help raise the performance ceiling, the baseline VDOM overhead remains an insurmountable bottleneck for massive density.

Conversely, Svelte 5 represents the paradigm shift required for high-performance WASM UI binding. Unlike React, Svelte operates purely as a compiler, eliminating the Virtual DOM entirely.³⁶ With the introduction of Svelte 5's Runes (specifically the `$state` and `$derived` primitives), reactivity is managed via fine-grained proxies rather than component-level re-rendering.⁴⁰ When a variable changes, Svelte's compiled code surgically targets the exact DOM text node or attribute associated with that variable, executing a direct assignment operation.⁴² This mechanism generates near-zero memory churn, as no intermediate tree structures are allocated, thereby completely evading GC triggers.

Binding high-frequency WASM state to Svelte 5 requires specific architectural patterns. Because Svelte's `$state` relies on object proxies to track mutations, directly exporting primitive values from a `SharedArrayBuffer` breaks reactivity.⁴⁴ Instead, engine architects wrap the WASM memory buffer reads within getter functions or derived state objects.⁴⁵ For instance, a continuous polling loop mapped to `requestAnimationFrame` can read the `SharedArrayBuffer` and immediately assign the new numeric values into a centralized Svelte `$state` object.⁴⁴ Because Svelte 5 updates surgically, feeding it hundreds of updates per second from a game engine will not re-render the surrounding DOM tree, maintaining a flawless 60 FPS response rate.⁴² Furthermore, Svelte's baseline runtime footprint of approximately 1.6 KB (compared to React's ~40 KB core) dramatically accelerates initial JavaScript parsing times, leaving significantly more CPU headroom for WASM compilation and asset loading during the game's boot phase.³⁸

However, even with surgical updates, pushing updates to the UI at the exact 60 FPS rate of the simulation kernel is often unnecessary and computationally wasteful. Humans cannot read changing numbers in an economic ledger 60 times a second. Therefore, high-performance engines implement aggressive debouncing and throttling algorithms that decouple the 60 FPS state mutation tick from a capped 15 FPS or 30 FPS UI render cycle. While the WASM simulation runs at full speed, the Web Worker running the UI logic only pulls from the `SharedArrayBuffer` and calculates diffs every 33ms or 66ms. This temporal decoupling halves or quarters the UI rendering overhead, reserving maximum CPU cycles for the C++ simulation and pathfinding algorithms.

Furthermore, UI virtualization (or "windowing") remains absolutely necessary regardless of the framework used. Even without VDOM overhead, simply existing in the DOM consumes heavy browser memory. If an empire ledger contains 10,000 provinces, forcing the browser to render 10,000 physical `<tr>` table rows will inevitably crash the layout engine. Virtualization intercepts the scroll event and mathematically calculates which 30 rows are currently visible within the viewport bounds.³¹ The DOM only ever contains those 30 physical elements. As the user scrolls, the data bound to those 30 elements is hot-swapped, and their absolute CSS `translate3d` positions are adjusted to simulate the visual effect of scrolling.⁴⁷ When combined with Svelte 5's surgical reactivity and render throttling, massive datasets can stream from the

WASM SAB into the virtualized viewport with microsecond latency and zero layout thrashing.

Decoupled Shadow State Management and Unidirectional Data Flow

Grand strategy engines fundamentally rely on deterministic simulations. Particularly for peer-to-peer multiplayer architectures, exchanging entire game states over the network is physically impossible due to bandwidth constraints. Instead, games utilize strict lockstep models where only a base initial random seed and the continuous stream of player inputs are exchanged across the network.⁴⁸ Every client's WASM engine must simulate the exact same ticks and arrive at the exact same mathematical state. Achieving this level of determinism requires absolute, impenetrable isolation between the simulation kernel and the UI presentation layer.⁵⁰

Floating-point operations defined by the IEEE 754 standard can produce microscopic variations across different CPU architectures and browser sandbox implementations. Over hundreds of thousands of simulation ticks, these microscopic variations compound, ultimately destroying lockstep synchronization and causing multiplayer desyncs.⁵¹ Consequently, the WASM simulation kernel must rely entirely on deterministic fixed-point arithmetic (e.g., Q16.16 formats) and 64-bit integer calculations for all gameplay-critical mathematics, pathfinding, and combat resolution.⁵¹ The UI, however, can safely interpolate this data into standard floating-point coordinates for visual smoothing and camera rendering.

To maintain these strict boundaries, the architecture must implement a decoupled Shadow State and leverage Hot/Cold Data Splitting. In a unified data struct—as advocated by engine designers like Casey Muratori—game entities contain both "hot" data (accessed frequently by the CPU for game logic, such as hit points, ammo, or map coordinates) and "cold" data (accessed only by the renderer, such as visual color tints, aesthetic particle pointers, or animation frame timers).⁵² The simulation kernel updates the hot data deterministically. The UI layer, however, maintains a strictly read-only relationship with this unified struct.⁵² It reads the state via the `SharedArrayBuffer` as a "shadow state"—a safely cached snapshot of the previous tick. The UI is completely isolated from the logic kernel, preventing any DOM event callback from inadvertently mutating a variable that could cause a temporal race condition or a multi-player desynchronization.

Because the UI cannot directly modify the game state, an Unidirectional Data Flow mechanism, heavily adapted from web paradigms like Flux and Redux, is mandatory.¹⁸ In traditional web development, a button click might immediately execute a state change. In a deterministic game loop, when a user interacts with the DOM—for instance, clicking a button to declare war or move a unit—this action does not immediately execute. Instead, the UI serializes this interaction into an intent object. This serialized intent is dispatched into a centralized pending Event Queue (or Intent Queue).⁵³

The Event Queue operates strictly on temporal boundaries. When the WASM engine begins its

next deterministic tick, it first drains the pending Intent Queue, processing all human inputs simultaneously alongside inputs received over the network from other clients.⁵⁴ This serialized, event-driven sequencer architecture ensures that inputs are processed in a mathematically rigid sequence. The result is total determinism: given the same inputs at the exact same tick, the WASM engine will invariably produce the exact same outputs, allowing high-frequency systems to process millions of operations reliably.⁵⁵ If the event queue were bypassed and the UI allowed to trigger engine functions asynchronously via standard JavaScript callbacks, the precise tick at which the action occurred would fluctuate based on the browser's main-thread workload, instantly destroying lockstep capability and rendering multiplayer impossible.⁵⁶

Rendering Density Thresholds: HTML DOM vs. Retained-Mode UI (Canvas)

Given the inherent performance ceilings of standard HTML rendering engines, a critical architectural decision in grand strategy development is identifying the precise density threshold at which the engine must abandon HTML/CSS and shift rendering to WebGL, Canvas, or Immediate-Mode GUI frameworks compiled directly into WebAssembly.

The DOM is unparalleled for accessibility, native screen-reader support, responsive scaling, and complex text formatting.³⁵ However, its underlying architecture—relying on individual C++ DOM node objects in browser memory, complex cascading style rules, and heavy layout flow algorithms—hits a hard performance limit around 2,000 to 5,000 simultaneously animating elements. Beyond this density, even with FastDOM batching, aggressive virtualization, and translate3d hardware acceleration, the sheer memory overhead of the node objects maintained by Blink or Gecko will drag the framerate below 60 FPS.³⁵

In stark contrast, WebGL and CanvasRenderingContext2D operate on a fundamentally different rendering paradigm. They manipulate generic pixels and raw JavaScript objects directly on the GPU, completely devoid of layout algorithms and individual node memory overhead.³⁵ A highly optimized WebGL context can comfortably render and animate well over 100,000 graphical entities simultaneously, making it strictly necessary for high-density data visualizations, full-scale 3D tactical maps, massive troop movement arrays, and complex shader-based particle systems.⁵⁷

For UI generation specifically, compiling an Immediate-Mode GUI framework like Dear ImGui directly into WebAssembly is highly attractive for developers seeking maximum performance and minimal integration friction. Dear ImGui operates by discarding UI state every frame and reconstructing the visual interface based on instantaneous game data, executing incredibly fast via WASM and bypassing the browser's slow DOM entirely.⁶¹ However, immediate-mode frameworks force the CPU to constantly redraw every single visual element every frame. For grand strategy software running physical, economic, and AI simulations that push CPU cores to 100% utilization, this constant redrawing steals vital processing cycles from the logic kernel.⁶⁴ Retained-mode DOM, conversely, caches its visual state; if a UI window doesn't change, the

browser simply composites the existing GPU texture, consuming zero CPU cycles and conserving battery life.³⁵ Furthermore, rendering text within WebGL or ImGui requires complex texture atlases and sacrifices native web accessibility features.

To resolve these conflicting constraints and maximize the strengths of both paradigms, modern grand strategy engines (such as Paradox Interactive's Jomini engine or Coherent Labs' Gameface) rely on deeply integrated hybrid strategies.⁶⁵ In these architectures, the primary engine layers—handling the complex 3D globe, topographical data, unit meshes, and dense particle effects—are strictly governed by WebGL/Canvas via WASM. However, seamlessly over-laid on top of this WebGL canvas is a transparent HTML5/CSS3 layer. All heavy text elements, dynamic ledgers, diplomatic menus, interactive buttons, and tooltips are rendered using the standard DOM.⁶⁸

This hybrid approach capitalizes on the specific strengths of both systems. Middleware like Coherent Gameface optimizes this integration by isolating the HTML layout calculations to background worker threads and highly batching rendering commands so only modified parts of the page are passed to the GPU texture.⁶⁵ By restricting HTML to layout-based menus and offloading all dense, high-frequency graphical map animations to WebGL, developers preserve standard web accessibility, ease of styling via CSS, and rich text rendering, while simultaneously achieving the extreme rendering density and performance required by the grand strategy genre.

Rendering Technology	Optimal Workload Profile	Maximum Animating Elements (Estimated)	CPU Usage Profile	Key Architectural Limitations
HTML/CSS DOM	Modals, Ledgers, Interactive Menus, Text-heavy tooltips.	~2,000 - 5,000	Low (Retained Mode, GPU Compositing handles static elements).	Severe layout thrashing risk; High memory overhead per individual node.
WebGL / Canvas	Tactical Maps, Unit Meshes, Particle Systems, High-density	100,000+	Variable (Highly dependent on Shader optimization and	Poor native text rendering (requires texture atlases); No native accessibility/scree

	graphs.		Draw Calls).	n-reader support.
Dear ImGui (WASM)	Developer Tools, Debug Overlays, Simple HUDs.	10,000+	High (Immediate Mode, requires constant redrawing every frame).	Steals vital CPU cycles from the simulation kernel; Lacks standard CSS styling flexibility.

Synthesis and Architectural Directives

Deploying a high-frequency, deterministic grand strategy game within the constraints of modern web browsers requires fundamentally circumventing the standard design paradigms of the JavaScript execution environment. To safely bind a massive state machine to the UI without stalling the rendering pipeline, exhausting memory, or causing temporal desynchronization, the architecture must abandon synchronous data serialization and strip the main thread of all business logic.

The optimal, production-grade architectural pattern demands a rigorous, multi-tiered approach. The core simulation must run deterministically in a Web Worker, utilizing strict fixed-point mathematics to avoid floating-point divergence. This logic must expose its state via a lockless `SharedArrayBuffer`, utilizing modern primitives like `Atomics.waitAsync` to orchestrate tick synchronization without thread blocking, while anticipating zero-copy text transfers via the emerging stringref specification. The presentation layer must implement a strict unidirectional data flow, processing all user interactions strictly as serialized intents that are queued for the subsequent engine tick, thereby preserving the lockstep determinism required for peer-to-peer multiplayer.

For the UI generation itself, Off-Main-Thread diffing combined with Svelte 5's surgical `$state` reactivity allows for massive data updates while totally evading V8 garbage collection pauses and Virtual DOM calculation bottlenecks. By decoupling the simulation rate from the UI render rate via throttling algorithms, and passing only primitive mutation instructions to the main thread batched via `requestAnimationFrame`, forced layout thrashing is eliminated. Finally, the integration must adopt a hybrid rendering philosophy—leveraging hardware-accelerated, virtualized HTML for accessible, text-heavy menus, while reserving WebGL canvases for the unbounded density of the strategic map. This synthesis of zero-copy memory, worker-thread diffing, and GPU-composited DOM structures provides the definitive blueprint for bypassing standard web limitations and achieving native-grade game engine performance directly within the browser.

Works cited

1. Shared Array Buffers With WASM - Blog, accessed April 16, 2026, https://ddr0.ca/blog-posts/13.Shared_Array_Buffers_With_WASM
2. SharedArrayBuffer: The Hidden Super-Primitive That's Reshaping the Future of WebAssembly, .NET & Parallel Runtime Architecture | by Jacob Mellor | Medium, accessed April 16, 2026, <https://medium.com/@jacobscottmellor/sharedarraybuffer-the-hidden-super-primitive-thats-reshaping-the-future-of-webassembly-net-e369e667f6e9>
3. What JavaScript SharedArrayBuffer Actually Lets You Do - Medium, accessed April 16, 2026, <https://medium.com/@AlexanderObregon/what-javascript-sharedarraybuffer-actually-lets-you-do-9589f449fd75>
4. About SharedArrayBuffer & Atomics | by Andrea Giammarchi - Medium, accessed April 16, 2026, <https://webreflection.medium.com/about-sharedarraybuffer-atomics-87f97ddfc098>
5. Using JavaScript SharedArrayBuffers and Atomics - Blog Title, accessed April 16, 2026, <https://blogtitle.github.io/using-javascript-sharedarraybuffers-and-atomics/>
6. Atomics.wait() - JavaScript - MDN Web Docs - Mozilla, accessed April 16, 2026, https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/wait
7. Javascript Synchronize SharedArrayBuffer to main thread - Stack Overflow, accessed April 16, 2026, <https://stackoverflow.com/questions/45439334/javascript-synchronize-sharedarraybuffer-to-main-thread>
8. Atomics.waitAsync() - JavaScript - MDN Web Docs, accessed April 16, 2026, https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/waitAsync
9. Unlock the Power of JavaScript Concurrency with Atomics.waitAsync() | by Art @dailysandbox | Medium, accessed April 16, 2026, <https://medium.com/@dailysandbox/unlock-the-power-of-javascript-concurrency-with-atomics-waitasync-ef78a3080cd1>
10. Atomics.notify() - JavaScript - MDN Web Docs, accessed April 16, 2026, https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/notify
11. Breaking the WASM/JS communication performance barrier - Hacker News, accessed April 16, 2026, <https://news.ycombinator.com/item?id=44656516>
12. Understanding WebAssembly text format - MDN Web Docs, accessed April 16, 2026, https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/Understanding_the_text_format
13. The State of WebAssembly – 2025 and 2026 - Uno Platform, accessed April 16, 2026, <https://platform.uno/blog/the-state-of-webassembly-2025-2026/>
14. stringref/proposals/stringref/Overview.md at main · WebAssembly/stringref - GitHub, accessed April 16, 2026,

- <https://github.com/WebAssembly/stringref/blob/main/proposals/stringref/Overview.md>
15. WebAssembly JavaScript builtins - WebAssembly | MDN, accessed April 16, 2026,
https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/JavaScript_builtins
 16. Improving WebAssembly load times with Zero-Copy deserialization · Blog - Wasmer, accessed April 16, 2026,
<https://wasmer.io/posts/improving-with-zero-copy-deserialization>
 17. Clarity on units of string · Issue #65 · WebAssembly/stringref - GitHub, accessed April 16, 2026, <https://github.com/WebAssembly/stringref/issues/65>
 18. Use web workers to run JavaScript off the browser's main thread ..., accessed April 16, 2026, <https://web.dev/articles/off-main-thread>
 19. An Advanced Guide to Web Workers in JavaScript for Performance-Heavy Tasks - Medium, accessed April 16, 2026,
https://medium.com/@sohail_saifi/an-advanced-guide-to-web-workers-in-javascript-for-performance-heavy-tasks-67d27b5c2448
 20. Idiomatic React Web Workers: Migrating off the main-thread. | by Jay Kariesch - Medium, accessed April 16, 2026,
<https://jaykariesch.medium.com/idiomatic-react-web-workers-migrating-off-the-main-thread-8afe284a86cf>
 21. An Introduction to Web Workers - Medium, accessed April 16, 2026,
<https://medium.com/@siobhanpmahoney/a-brief-introduction-to-web-workers-e5d6e39d9d28>
 22. Get off the main thread (and fix INP) with inline web workers - Web ..., accessed April 16, 2026,
<https://calendar.perfplanet.com/2022/get-off-the-main-thread-with-an-inline-web-worker-an-example/>
 23. An overview of web workers - web.dev, accessed April 16, 2026,
<https://web.dev/learn/performance/web-worker-overview>
 24. Instructions — WebAssembly 3.0 (2026-04-08), accessed April 16, 2026,
<https://webassembly.github.io/spec/core/binary/instructions.html>
 25. Optimizing Performance with Web Workers - Salesforce Engineering Blog, accessed April 16, 2026,
<https://engineering.salesforce.com/optimizing-performance-with-web-workers-612b48621d8d/>
 26. JavaScript in parallel - web workers explained - DEV Community, accessed April 16, 2026,
<https://dev.to/g33konaut/javascript-in-parallel-web-workers-explained-5588>
 27. Layout thrashing: what is it and how to eliminate it - DEV Community, accessed April 16, 2026,
https://dev.to/aayla_secura/layout-thrashing-what-is-it-and-how-to-eliminate-it-n2j
 28. Layout Thrashing and Forced Reflows - Web Performance Tips, accessed April 16, 2026, <https://webperf.tips/tip/layout-thrashing/>
 29. What is a Forced Reflow and How to Solve it? - Yonatan Kra, accessed April 16,

- 2026, <https://yonatankra.com/layout-reflow/>
30. FastDom: eliminate layout thrashing by batching DOM measurement and mutation tasks, accessed April 16, 2026, <https://ambientimpact.com/web/snippets/fastdom-eliminate-layout-thrashing-by-batching-dom-measurement-and-mutation-tasks>
 31. Performance Optimization in DOM Manipulation - CarlosRojasDev, accessed April 16, 2026, <https://stories.carlosrojas.dev/2025/08/26/performance-optimization-in-dom-manipulation/>
 32. Browsers benchmark - #28 by Bink - Applications - EndeavourOS Forum, accessed April 16, 2026, <https://forum.endeavouros.com/t/browsers-benchmark/74608/28>
 33. Are There Any Objective Performance Differences Between Blink and Gecko? - Reddit, accessed April 16, 2026, https://www.reddit.com/r/browsers/comments/1jbcyy3/are_there_any_objective_performance_differences/
 34. Web performance - MDN Web Docs - Mozilla, accessed April 16, 2026, <https://developer.mozilla.org/en-US/docs/Web/Performance>
 35. Performance Difference Between DOM vs Canvas using Greensock - GSAP, accessed April 16, 2026, <https://gsap.com/community/forums/topic/10487-performance-difference-between-dom-vs-canvas-using-greensock/>
 36. Svelte vs. React: Choosing the best for features and performance - Contentful, accessed April 16, 2026, <https://www.contentful.com/blog/svelte-vs-react/>
 37. 99.9% of front-end work is only performance-critical for DOM rendering There a... | Hacker News, accessed April 16, 2026, <https://news.ycombinator.com/item?id=24122305>
 38. Svelte vs React: A Comprehensive Comparison for Developers - Strapi, accessed April 16, 2026, <https://strapi.io/blog/svelte-vs-react-comparison>
 39. Svelte or React? 10 Key Factors to Help You Decide in 2025 - SVAR UI, accessed April 16, 2026, <https://svar.dev/blog/svelte-vs-react/>
 40. \$state • Svelte Docs, accessed April 16, 2026, [https://svelte.dev/docs/svelte/\\$state](https://svelte.dev/docs/svelte/$state)
 41. Svelte 5 migration guide, accessed April 16, 2026, <https://svelte.dev/docs/svelte/v5-migration-guide>
 42. Real-world Svelte 5: Handling high-frequency real-time data with Runes - DEV Community, accessed April 16, 2026, <https://dev.to/polliog/real-world-svelte-5-handling-high-frequency-real-time-data-with-runes-3i2f>
 43. Help me love Svelte 5's reactivity – what am I missing about Maps and snapshots? - Reddit, accessed April 16, 2026, https://www.reddit.com/r/sveltejs/comments/1nawevv/help_me_love_svelte_5s_reactivity_what_am_i/
 44. Svelte 5: Share state between components (for Dummies) - DEV Community, accessed April 16, 2026,

- <https://dev.to/mandrasch/svelte-5-share-state-between-components-for-dummies-4gd2>
45. How to share reactive state across multiple components in Svelte 5? #17603 - GitHub, accessed April 16, 2026, <https://github.com/sveltejs/svelte/discussions/17603>
 46. Svelte 5 Patterns (Part 1): Simple Shared State, getContext, and Tweened Stores with \$runes - fubits, accessed April 16, 2026, <https://fubits.dev/notes/svelte-5-patterns-simple-shared-state-getcontext-tweened-stores-with-runes/>
 47. Benchmarking Frontends in 2025 : r/javascript - Reddit, accessed April 16, 2026, https://www.reddit.com/r/javascript/comments/1mti52z/benchmarking_frontends_in_2025/
 48. GitHub - turtle261/gameengine: A formally verified, deterministic, reversible game/simulation kernel designed as the reference environment layer for Infotheory., accessed April 16, 2026, <https://github.com/turtle261/gameengine>
 49. Preparing your game for deterministic netcode - YellowAfterlife, accessed April 16, 2026, <https://yal.cc/preparing-your-game-for-deterministic-netcode/>
 50. Game engine determinism (not just physics) : r/gamedev - Reddit, accessed April 16, 2026, https://www.reddit.com/r/gamedev/comments/17lg9yi/game_engine_determinism_not_just_physics/
 51. Lessons from building a browser-native RTS engine in 100K lines of TypeScript — deterministic lockstep, WebGPU rendering, and P2P multiplayer (open source, contributors welcome) : r/gamedev - Reddit, accessed April 16, 2026, https://www.reddit.com/r/gamedev/comments/1rtpanq/lessons_from_building_a_browser_native_rts_engine/
 52. Casey Muratori's Advices on Game Dev | by alex_ber | Medium, accessed April 16, 2026, <https://alex-ber.medium.com/casey-muratoris-advices-on-game-dev-fcdeb3fc7da7>
 53. Deterministic Game Engine: Practical Validation of Pointer-Based Security and Local Data Regeneration Paradigms - DEV Community, accessed April 16, 2026, <https://dev.to/smartlegionlab/deterministic-game-engine-practical-validation-of-pointer-based-security-and-local-data-408l>
 54. Event Queue - Game Programming Patterns, accessed April 16, 2026, <https://gameprogrammingpatterns.com/event-queue.html>
 55. The Deterministic Event-Driven Sequencer Architecture: A Competitive Edge for High-Frequency Trading | by Wenzhe Hu | Medium, accessed April 16, 2026, <https://medium.com/@hu.wenzhe124124/the-deterministic-event-driven-sequence-r-architecture-a-competitive-edge-for-high-frequency-371cbfbe9c2f>
 56. what are some game design patterns that you wish you knew earlier? : r/gamedev - Reddit, accessed April 16, 2026, https://www.reddit.com/r/gamedev/comments/10qtoe4/what_are_some_game_design_patterns_that_you_wish/
 57. Seating Plan Rendering: SVG vs Canvas vs WebGL - Seatmap.pro, accessed April 16, 2026, <https://seatmap.pro/blog/seating-plan-rendering/>

58. Single Page Applications using Rust - Hacker News, accessed April 16, 2026, <https://news.ycombinator.com/item?id=24120311>
59. Canvas rendering optimization - javascript - Stack Overflow, accessed April 16, 2026, <https://stackoverflow.com/questions/40068953/canvas-rendering-optimization>
60. High-Performance WebGL-Based Visual Analytics Framework for Large-Scale Behavioral Embeddings: System Architecture and Rendering Optimization - MDPI, accessed April 16, 2026, <https://www.mdpi.com/2076-3417/16/7/3307>
61. I was understanding WASM all wrong! | by Yuji Isobe - Medium, accessed April 16, 2026, <https://medium.com/@yujiisobe/i-was-understanding-wasm-all-wrong-e4bcab8d077c>
62. Recently I've been playing around with Dear ImGui and it's so fast that now all ... - Hacker News, accessed April 16, 2026, <https://news.ycombinator.com/item?id=40548714>
63. Why Use Dear ImGui for Game Engines if It's Not for Complex Apps? - Reddit, accessed April 16, 2026, https://www.reddit.com/r/cpp/comments/1erq2vx/why_use_dear_imgui_for_game_engines_if_its_not/
64. [Question] What's the actual impact on overall performance? imgui suitable for HPC software? · Issue #2703 - GitHub, accessed April 16, 2026, <https://github.com/ocornut/imgui/issues/2703>
65. Technical Overview - Gameface documentation - Coherent Labs, accessed April 16, 2026, <https://docs.coherent-labs.com/cpp-gameface/integration/technicaloverview/>
66. Jomini.js - Parses Paradox files into javascript objects - GitHub, accessed April 16, 2026, <https://github.com/nickbabcock/jomini>
67. A Tour of PDS Clausewitz Syntax - PDX Tools, accessed April 16, 2026, <https://pdx.tools/blog/a-tour-of-pds-clausewitz-syntax>
68. How does Paradox Interactive make the sophisticated-looking GUIs they use in their games? : r/gamedev - Reddit, accessed April 16, 2026, https://www.reddit.com/r/gamedev/comments/42onxh/how_does_paradox_interactive_make_the/
69. Anyone experience with Coherent Labs Gameface? We want to go the HTML-Route for UI., accessed April 16, 2026, https://www.reddit.com/r/gamedev/comments/rlbsmu/anyone_experience_with_coherent_labs_gameface_we/
70. Technical Overview - Gameface documentation - Coherent Labs, accessed April 16, 2026, <https://docs.coherent-labs.com/unity-gameface/integration/technicaloverview/>